最优化方法程序作业
实验报告

吴羽迪 2300010631

目录

1 问题描述 **2**

2 评价指标 **2**

3 问题求解 **3**

3.1 CVX 直接求解原问题 3

3.2 直接调用 Mosek 和 Gurobi 求解原问题 4

3.3 连续化策略 5

3.4 次梯度算法求解原问题 6

3.5 梯度下降法求解原问题 6

3.6 Nesterov 加速梯度下降法求解原问题 9

3.7 近似梯度下降法求解原问题 10

3.8 FISTA 求解原问题 13

3.9 增广拉格朗日函数法求解对偶问题 13

3.10 交替方向乘子法求解对偶问题 15

3.11 交替方向乘子法求解原问题 16

4 实验结果 **18**

5 参考资料 **18**

1 问题描述

算法需要求解的 Group LASSO 问题为：

$$\min_{x\in\mathbb{R}^{n\times l}} \frac{1}{2}\|Ax-b\|_F^2 + \mu\|x\|_{1,2} \tag{1}$$

其中 $A\in\mathbb{R}^{m\times n}$, $b\in\mathbb{R}^{m\times l}$, $\mu>0$ 并且

$$\|x\|_{1,2} = \sum_{i=1}^{n} \|x_{(i,1:l)}\|_2$$

其中 $x_{(i,1:l)}$ 是矩阵 x 的第 i 行。

2 评价指标

项目完成了多个算法的实现。为了实现不同算法效率之间的对比，本项目将使用以下指标来评价各个算法的性能。

- `Err-to-exact`：算法得到的最终目标函数值和最优函数值之间的差的绝对值：

  $$\texttt{Err-to-exact}(f_x, f_u) = |f_x - f_u|$$

  其中 f_x 为算法得到的最优函数值，f_u 为实际最优函数值（在本次实验中，f_u 取为 CVX 调用 Mosek 或 Gurobi 求解器求解此 Group Lasso 问题时，得到的结果）。

- `Err-to-CVX-Mosek/Gurobi`：算法得到的最终解和最优解之间的归一化范数差：

  $$\texttt{Err-to-CVX-Mosek/Gurobi}(x, u) = \frac{\|x-u\|_F}{1+\|u\|_F}$$

  其中 x 为算法得到的最优解，u 为实际最优解（在本次实验中 u 取为 CVX 调用 Mosek 或 Gurobi 求解器求解此 Group Lasso 问题时，得到的结果）。

- `Sparasity`：算法得到的最终解的稀疏度：

  $$\texttt{Sparasity} = \frac{\|x\|_0}{n\times l}$$

其中 x 为算法得到的最终解，$\|x\|_0$ 表示 x 中非零元素的个数。在本项目中认为绝对值小于 10^{-6} 的元素为 0。

- `Optval`：算法得到的最终目标函数值。
- `CPU(s)`：算法求解的运行时间。
- `Iter`：算法求解的迭代次数。

3 问题求解

3.1 CVX 直接求解原问题

在安装好 Mosek 和 Gurobi 求解器后，我们可以直接通过调用 Matlab 中的 CVX 工具包，使用这两个求解器进行求解。求解器的运行日志分别保存在 `gl_cvx_mosek.txt` 和 `gl_cvx_gurobi.txt` 中，从日志中读取数据后可绘图1如下：

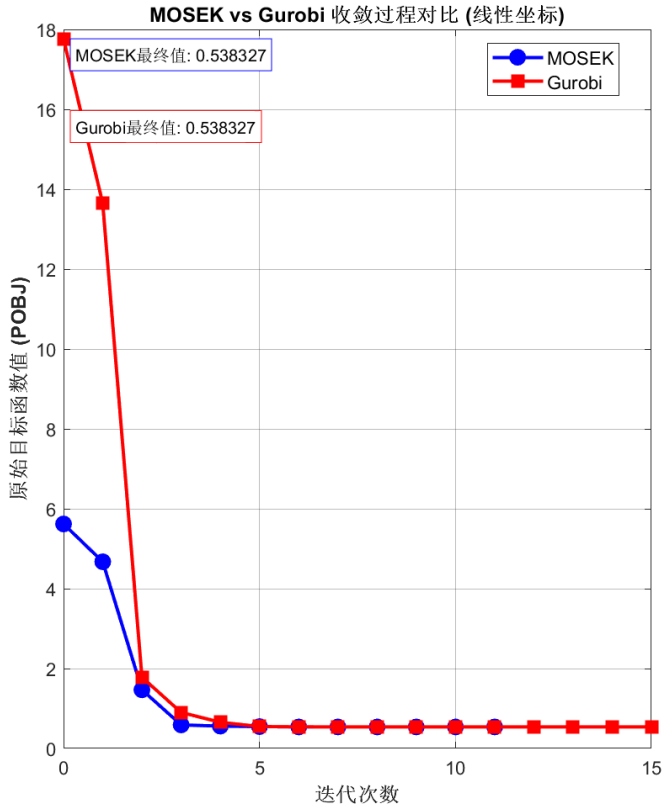


图 1: mosek 和 gurobi 收敛过程对比

从图1以及表1中可知，使用 Mosek 求解的迭代次数比使用 Gurobi 求解的少，但是 Gurobi 刚开始时收敛的更快，求解的时间也更少。同时，Mosek 和 Gurobi 得到的解在数值上几乎相同，稀疏度也基本一致。

3.2 直接调用 Mosek 和 Gurobi 求解原问题

Mosek 和 Gurobi 有对应的 matlab 软件包，可以直接调用对应的求解函数进行求解，但需要先将原本的 Group Lasso 问题转化为函数接口要求的标准形式，如下：

$$\begin{aligned}\min_{x,t,y,u} \quad & u+\mu\sum_{i=1}^{n}t_i \\ \text{s.t.} \quad & Ax-b=y \\ & \frac{1}{2}\|y\|_F^2\le u \\ & \|x(i,:)\|_2\le t_i, \quad i=1,\dots,n\end{aligned}$$

倘若我们记 x,b,y 的第 i 列分别为 $x_i,b_i,y_i(1\le i\le l)$，并记

$$vec=(x_1^T,\dots,x_l^T,y_1^T,\dots,y_l^T,u,1,t_1,\dots,t_n)^T\in\mathbb{R}^{M\times 1}$$

其中 $M=(n+m)l+n+2$，则上述优化问题可转化为：

$$\begin{aligned}\min_{vec} \quad & c^T vec \\ \text{s.t.} \quad & \tilde{A}\cdot vec=-b \\ & vec(nl+1:(n+m)l+1)\in\mathcal{Q}^{ml+1} \\ & (vec((i-1)l+1:il),vec((n+m)l+2),vec((n+m)l+2+i))\in\mathcal{Q}_r^{l+2},\ i=1,\dots,n\end{aligned}$$

其中 $\mathcal{Q},\mathcal{Q}_r$ 分别为二次锥和旋转二次锥，且

$$c=(0,\dots,0,1,0,\mu,\dots,\mu)^T\in\mathbb{R}^{nM\times 1}$$

$$\tilde{A}=(diag\{-A,\dots,-A\},diag\{I_m,\dots,I_m\},\mathbf{0}^{m\times(n+2)})\in\mathbb{R}^{m\times M}$$

此时已经转化为 Mosek 和 Gurobi 求解器要求的格式，调用求解器即可完成优化求解，结果如图2

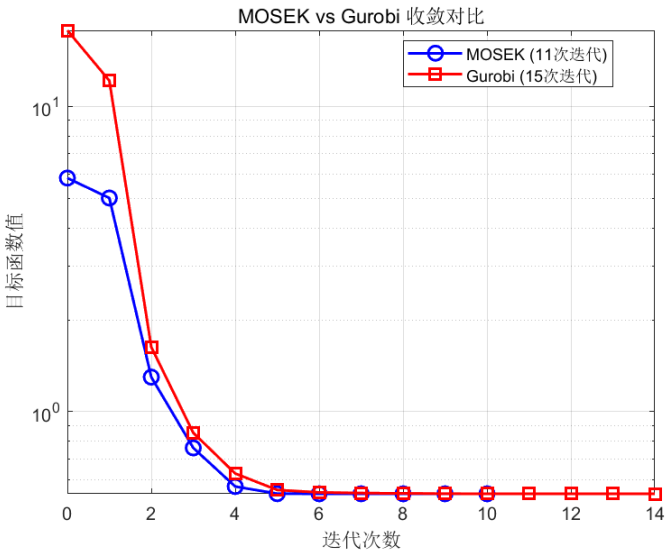


图 2: mosek 和 gurobi 收敛过程对比

由图2及表1可知，直接调用 Mosek 的求解效率比直接调用 Gurobi 的更快，迭代次数更少；使用 CVX 调用两个求解器比直接调用两个求解器用时更长，可能 CVX 在转换问题形式上消耗了更多时间；直接调用与使用 CVX 调用在结果以及迭代次数上相差无几。

3.3 连续化策略

除了 ALM-dual 和 ADMM-dual，接下来的算法都使用了连续化策略。对于本项目解决的 Group Lasso 问题，连续化策略从较大的正则化参数 μ_t 逐步减小到 μ（即 $\mu_1 \geq \cdots \geq \mu_{t-1} \geq \mu_t \geq \cdots \geq \mu$），并求解相应的 Group Lasso 问题：

$$\min_{x \in \mathbb{R}^{n \times l}} \frac{1}{2}\|Ax - b\|_F^2 + \mu_t \|x\|_{1,2}$$

这样做的好处是：在求解 μ_t 对应的优化问题时，可以利用 μ_{t-1} 对应的优化问题的解（μ_1 子问题使用随机初始点）作为一个很好的逼近解以在较短的时间内完成求解过程；μ_t 越大，相应的 Group Lasso 问题越好解。因此，连续化策略相当于是通过快速求解一系列简单问题来加速求解原始问题。

正则化参数的更新公式为 $\mu_{t+1} = max\{\mu_0, \mu_t \eta\}$，其中 $\eta \in (0, 1)$ 为缩小因子。在代码实现中，最初的正则化参数是 100μ 或 1000μ，每次乘 0.1，两或三次后得到 μ。

3.4 次梯度算法求解原问题

次梯度算法的具体流程如下述 **Algorithm 1**。需要注意的是，在本项目的代码实践中，倘若某个向量的 2-范数小于 $1e-6$，我们认为该向量为零向量，并且取它的次梯度为零向量。

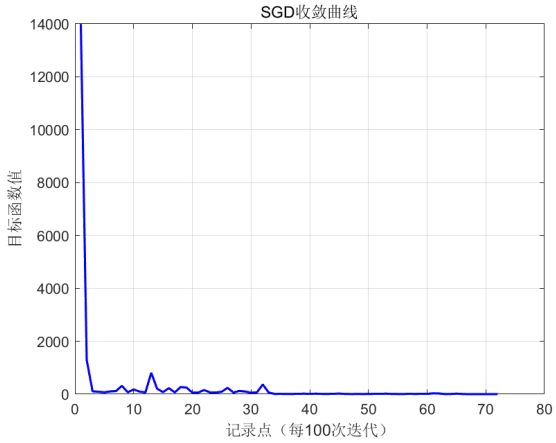


图 3: SGD 收敛过程

算法中提到的 f^* 为先验的函数最优值，我们使用的是之前 mosek 求解器求得的函数最优值。由表1和图3可知，SGD 算法的收敛速度较慢，迭代次数较多，但是每次迭代的时间更短，总用时较为适中。此外，相较于其他算法，SGD 的收敛曲线较为震荡。这可能是因为步长的选取有时太大，但函数值的曲线最终是收敛到最优值的。

3.5 梯度下降法求解原问题

由于 $\|x\|_{1,2}$ 在 $x=0$ 处是不可微的，我们需要将原本的 Group Lasso 问题光滑化，这样才能求出梯度，使用梯度下降法。由于对任意的 l 维列向量 x，有

$$\|x\|_2 = \max_{\|y\|_2 \le 1} y^T x$$

所以

$$\|x\|_{1,2} = \max_{Y \in \mathcal{Y}} \langle x, Y \rangle$$

其中 $\mathcal{Y} = \{Y \in \mathbb{R}^{n \times l} : \ \|Y(i,:)\|_2 \le 1, \ i = 1, 2, \ldots, n\}$

Algorithm 1: 次梯度法

Input: $A \in \mathbb{R}^{m\times n}, b \in \mathbb{R}^{m\times l}, \mu > 0$，最大迭代次数 max_iter，阈值 $tol > 0$，最优函数值 f^*

Initialize $x^{(0)}$

for $k = 0, 1, 2, \ldots, max_iter$ **do**

检查停机条件：

$$if \quad f(x^{(k)}) - f^* < tol, \quad break$$

计算次梯度 $g^{(k)}$:

$$g^{(k)} = A^T(Ax^{(k)} - b) + \mu \cdot \partial_{\ell_{1,2}}(x^{(k)})$$

计算步长 $\alpha^{(k)}$:

$$\alpha^{(k)} = \frac{f(x^{(k)}) - f^*}{\|g^{(k)}\|_2^2}$$

更新 $x^{(k+1)}$:

$$x^{(k+1)} = x^{(k)} - \alpha^{(k)} g^{(k)}$$

end

return $x^{(k+1)}$

使用光滑参数 $\delta > 0$，邻近函数

$$d(Y) = \frac{1}{2}\|Y\|_F^2 = \frac{1}{2}\sum_{i=1}^{n}\|Y(i,:)\|_2^2$$

记 $x_i = x(i,:), y_i = Y(i,:)$，则有

$$\begin{aligned}\|x\|_{1,2,\delta} &= \max_{Y\in\mathcal{Y}}\langle x, Y\rangle - \delta d(Y) \\ &= \max_{\|y_i\|_2\le 1}\sum_{i=1}^{n}\langle x_i, y_i\rangle - \frac{\delta}{2}\|y_i\|_2^2 \\ &= \sum_{i=1}^{n} f_\delta(\|x_i\|_2)\end{aligned}$$

其中

$$f_\delta(x) = \begin{cases} \dfrac{t^2}{2\delta}, & t \le \delta \\ t - \dfrac{\delta}{2}, & t > \delta \end{cases}$$

原问题 (1) 光滑化为

$$\min_{x\in\mathbb{R}^{n\times l}} \frac{1}{2}\|Ax - b\|_F^2 + \mu\|x\|_{1,2\,\delta} \tag{2}$$

其中 μ 为正则化参数，δ 为光滑化参数。对 (2) 中的目标函数关于 x 求导，可得梯度矩

Algorithm 2: 梯度下降法

Input: $A \in \mathbb{R}^{m\times n}, b \in \mathbb{R}^{m\times l}, \mu > 0$，最大迭代次数 max_iter，阈值 $tol > 0$，δ 的起始与终止值 $\delta_{start} > \delta_{final} > 0$, 递减步数 K

Initialize $x^{(0)}$

for $k = 0, 1, 2, \ldots, K$ **do**

$$\delta = \delta_{start} - k \times \frac{\delta_{start} - \delta_{final}}{K}$$

  for $i = 0, 1, 2, \ldots, max_iter$ **do**

    检查停机条件：

$$if \quad \|g_\delta(x^{(i)})\|_F^2 < tol, \quad break$$

    计算步长：

    if $i > 0$ **then**

$$s^{(i-1)} = x^{(i)} - x^{(i-1)}$$

$$y^{(i-1)} = y^{(i)} - y^{(i-1)}$$

$$\alpha^{(i)} = \frac{(s^{(i-1)})^T s^{(i-1)}}{(s^{(i-1)})^T y^{(i-1)}}$$

    end

    else

$$\alpha^{(i)} = \frac{1}{\|A\|_2^2 + \frac{\mu}{\delta}}$$

    end

    更新 $x^{(i+1)}$:

$$x^{(i+1)} = x^{(i)} - \alpha^{(i)} g_\delta(x^{(i)})$$

  end

  $x^{(0)} = x^{(i+1)}$

end

return $x^{(0)}$

阵为

$$g_\delta(x) = A^T(Ax - b) + \mu G$$

其中 G 的第 i 行为

$$\frac{\alpha}{\alpha x_i}\|x\|_{1,2,\delta} = \begin{cases} \dfrac{x_i}{\delta}, & \|x_i\|_2 \le \delta \\ \dfrac{x_i}{\|x_i\|_2}, & \|x_i\|_2 > \delta \end{cases}$$

我们使用 BB 步长进行更新。然而，由于每次构造 BB 步长需要上一步的数据，所以第一步无法使用 BB 步长。于是在第一步我们取步长为 $\frac{1}{L}$，其中 $L = \|A\|_2^2 + \frac{\mu}{\delta}$ 是 $g_\delta(x)$ 的 *Lipschitz* 常数。

下面我们给出具体的算法 **Algorithm 2** ，用此算法解决 Group Lasso 问题时，函数值的变化如图4

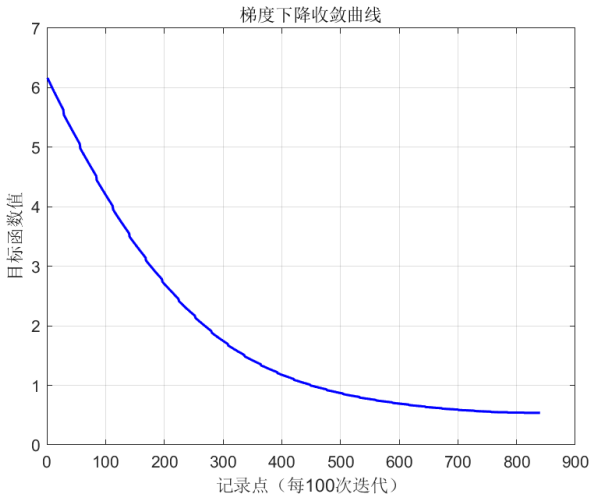


图 4: GD 收敛过程

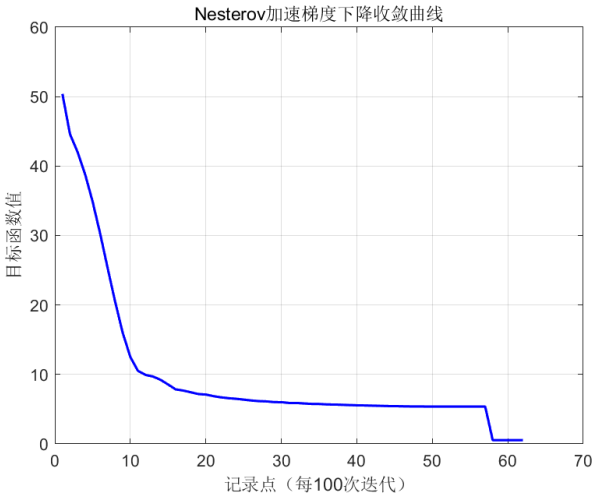


图 5: FGD 收敛过程

从表1和图4中可以看出，梯度下降所用的迭代次数和时间都是最多的。在代码实现的过程中，我发现如果想用梯度下降法更加精确地求解，所需要的时间会大大增加。在权衡取舍下，最终求解出的结果和最优值有一定差距，但是运行时间较为合适。

3.6 Nesterov 加速梯度下降法求解原问题

Nesterov 加速梯度下降法几乎相同，只是计算步长的方式不同，算法 **Algorithm 3**如下。在此算法中，我们不再采用 BB 步长，而是采用固定步长 $\frac{1}{L}$。用此算法解决 Group Lasso 问题时，函数值的变化如图5。

相较于 GD，FGD 用时更少，迭代次数也更少，可以适当牺牲时间来得到比 GD 更精确的结果，如表1所示。图5中下降较快的一个阶梯状部分是 μ 变小导致的结果。

Algorithm 3: Nesterov 加速梯度下降法

Input: $A \in \mathbb{R}^{m\times n}, b \in \mathbb{R}^{m\times l}, \mu > 0$，最大迭代次数 max_iter，阈值 $tol > 0$，δ 的起始与终止值 $\delta_{start} > \delta_{final} > 0$, 递减步数 K

Initialize $x^{(0)}$

for $k = 0, 1, 2, \ldots, K$ **do**

$$\delta = \delta_{start} - k \times \frac{\delta_{start} - \delta_{final}}{K}$$

$$L = \|A\|_2^2 + \frac{\mu}{\delta}$$

  for $i = 0, 1, 2, \ldots, max_iter$ **do**

    检查停机条件：

$$if \quad \|g_\delta(x^{(i)})\|_F^2 < tol, \quad break$$

    计算步长：

    if $i > 0$ **then**

$$\beta^{(i)} = \frac{i-1}{i+2}$$

$$y^{(i)} = x^{(i)} + \beta^{(i)}(x^{(i)} - x^{(i-1)})$$

$$x^{(i+1)} = y^{(i)} - \frac{1}{L} \times g_\delta(y^{(i)})$$

    end

    else

$$x^{(i+1)} = x^{(i)} - \frac{1}{L} \times g_\delta(x^{(i)})$$

    end

  end

  $x^{(0)} = x^{(i+1)}$

end

return $x^{(0)}$

3.7 近似梯度下降法求解原问题

对于复合优化问题：

$$\min_x \phi(x) + h(x)$$

其中 ϕ 光滑，h 非光滑。近似点梯度法的迭代格式为

$$x^{(k+1)} = prox_{t^{(k)}h(\cdot)}(x^{(k)} - t^{(k)}\nabla\phi(x^{(k)}))$$

这里 $t^{(k)}$ 是步长

在 Group Lasso 问题中，有

$$\phi(x) = \frac{1}{2}\|Ax - b\|_F^2$$
$$h(x) = \mu\|x\|_{1,2}$$
$$\nabla\phi(x) = A^T(Ax - b)$$

下面来求解近似点算子的解析解。近似点算子的定义为

$$prox_{th}(y) = \arg\min_x\{h(x) + \frac{1}{2t}\|x - y\|_F^2\}$$

对于 $h(x) = \mu\|\cdot\|_{1,2}$，由于惩罚项是逐行可分的，我们可以对每一行独立求解。设 y_i 为输入矩阵的第 i 行（长度为 l），则需求解

$$prox_{t\mu\|\cdot\|_2}(y_i) = \arg\min_{x_i}\{\mu\|x_i\|_2 + \frac{1}{2t}\|x_i - y_i\|_2^2\}$$

这是一个凸问题，最优解满足一阶最优性条件。该问题的解为

$$prox_{th}(y_i) = \max(0, 1 - \frac{t\mu}{\|y_i\|_2})y_i$$

推导如下：

- $x_i \neq 0$，目标函数可微。令梯度为零：

$$\mu\frac{x_i}{\|x_i\|_2} + \frac{1}{t}(x_i - y_i) = 0$$

整理得：

$$x_i = y_i - \frac{t\mu}{\|x_i\|_2}x_i$$

由此可得 x_i 与 y_i 同向。设 $x_i = \alpha y_i(\alpha \geq 0)$，代入上式解得：

$$\alpha = 1 - \frac{t\mu}{\|y_i\|_2}$$

由于要求 $\alpha > 0$，此时须，满足 $\|y_i\|_2 > t\mu$。

Algorithm 4: 近似梯度下降法

Input: $A \in \mathbb{R}^{m\times n}, b \in \mathbb{R}^{m\times l}, \mu > 0$，最大迭代次数 max_iter，阈值 $tol > 0$

Initialize $x^{(0)}$

for $k = 0, 1, 2, \ldots, max_iter$ **do**

计算梯度 $g^{(k)}$:

$$g^{(k)} = A^T(Ax^{(k)} - b)$$

计算近似点算子

$$x_{\text{prox}} = \text{prox}_{\frac{\mu}{L}\|\cdot\|_{1,2}}\left(x^{(k)} - \frac{1}{L}g^{(k)}\right)$$

更新 $x^{(k+1)}$:

$$x^{(k+1)} = x_{\text{prox}}$$

检查停机条件：

$$if \quad \|x^{(k+1)} - x^{(k)}\|_2 < tol, \quad break$$

end

return $x^{(k+1)}$

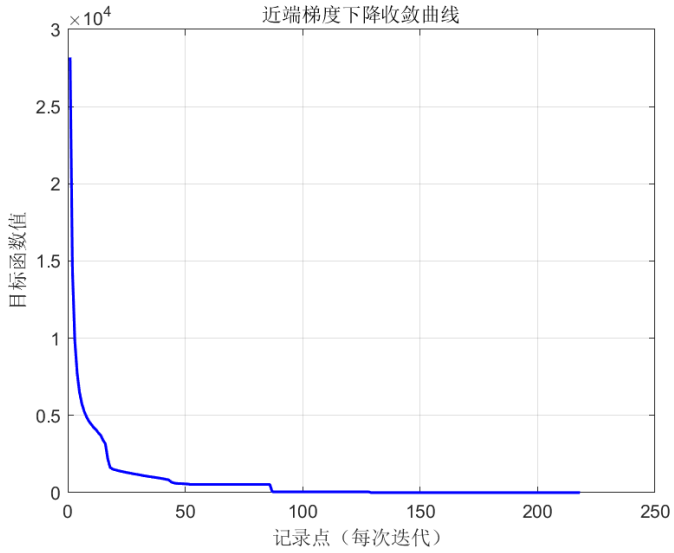


图 6: ProxGD 收敛过程

- $x_i = 0$，次梯度条件为存在 $z \in \partial(\|\cdot\|_2)(0)$（即 $\|z\|_2 \le 1$）使得：

$$0 = \mu z - \frac{1}{t}y_i \Rightarrow y_i = t\mu z$$

因此需满足条件 $\|y_i\|_2 \le t\mu$

我们记 $L = \|A\|_2^2$，并在近似梯度下降法中取固定步长 $\frac{1}{L}$，整理得到的算法 **Algorithm 4**如下；用近似梯度下降法解决 Group Lasso 问题时，函数值的变化如图6

从表1中可以看出，相比前面三种梯度下降，ProxGD 的求解速度较快，迭代次数较少。从图6中可以看出，函数值刚开始时下降极快。这是使用连续化的好处：我们可以先使用较大的 μ 快速减小函数值。

3.8 FISTA 求解原问题

Nesterov 加速近似梯度下降法与近似梯度下降法类似，只是更新的过程稍有不同，给出算法 **Algorithm 5**如下，运行结果如图7

Algorithm 5: FISTA

Input: $A \in \mathbb{R}^{m\times n}, b \in \mathbb{R}^{m\times l}, \mu > 0$，最大迭代次数 max_iter，阈值 $tol > 0$
Initialize $x^{(0)}$
for $k = 0, 1, 2, \ldots, max_iter$ **do**
计算近似点算子

$$y^{(k)} = x^{(k)} + \frac{k-1}{k+2}(x^{(k)} - x^{(k-1)})$$
$$g^{(k)} = A^T(Ay^{(k)} - b)$$
$$x_{\text{prox}} = \text{prox}_{\frac{\mu}{L}\|\cdot\|_{1,2}}\left(y^{(k)} - \frac{1}{L}g^{(k)}\right)$$

更新 $x^{(k+1)}$:

$$x^{(k+1)} = x_{\text{prox}}$$

检查停机条件：

$$if \quad \|x^{(k+1)} - x^{(k)}\|_2 < tol, \quad break$$

end
return $x^{(k+1)}$

FProxGD 和 PorxGD 的求解结果相似，相比之下前者用时更短，但所需迭代次数更多。两者的收敛曲线也极为相似，都是一条非常贴近两条坐标轴的曲线。

3.9 增广拉格朗日函数法求解对偶问题

先给出原问题的对偶问题：

$$\begin{aligned} \min_{z} \quad & \frac{1}{2}\|z\|_F^2 + \langle b, z\rangle \\ \text{s.t.} \quad & \|(A^T z)_i\|_2 \le \mu,\ i = 1, 2, \ldots, n \end{aligned}$$

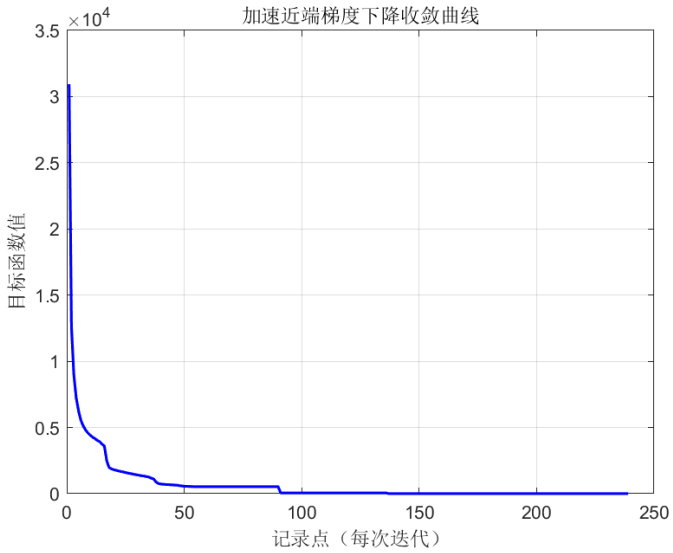


图 7: FProxGD 收敛过程

其中 $(A^Tz)_i$ 是 A^Tz 的第 i 行（z 是 $m \times l$ 矩阵，A^Tz 是 $n \times l$ 矩阵）。引入辅助变量，对偶问题等价于

$$\begin{aligned} \min_{z} \quad & \frac{1}{2}\|z\|_F^2 + \langle b, z\rangle \\ \text{s.t.} \quad & A^Tz = w \\ & \|w_i\|_2 \le \mu,\ i = 1, 2, \ldots, n \end{aligned}$$

引入拉格朗日乘子 $\lambda \in \mathbb{R}^{n\times l}$ 和罚因子 $\sigma > 0$，作增广拉格朗日函数：

$$L_\sigma(z, w, \lambda) = \frac{1}{2}\|z\|_F^2 + \langle b, z\rangle + \langle \lambda, A^Tz - w\rangle + \frac{\sigma}{2}\|A^Tz - w\|_F^2, \quad \|w\|_{\infty,2} \le \mu$$

增广拉格朗日函数法的迭代格式为：

$$\begin{cases} (z^{k+1}, w^{k+1}) = \arg\min\limits_{z, \|w\|_{\infty,2}\le\mu} L_{\sigma_k}(z, w, \lambda^k), \\ \lambda^{k+1} = \lambda^k + \sigma_k(A^Tz^{k+1} - w^{k+1}), \\ \sigma_{k+1} = \min\{\rho\sigma_k, \bar{\sigma}\}. \end{cases}$$

先求第一个式子，当 $z^{k+1} = z$ 固定时，解得 w^{k+1} 为：

$$w^{k+1} = \mathcal{P}_{\|w\|_{\infty,2}\le\mu}(A^Tz + \frac{\lambda^k}{\sigma_k})$$

将上式代回第一个式子，并用梯度下降法，可以求得 z^{k+1}，进而求得 w^{k+1}。此算法的运行结果如图8

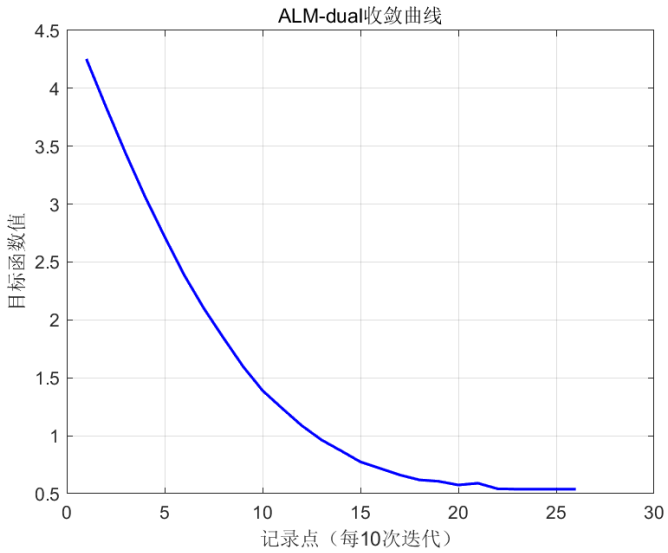


图 8: ALM-dual 收敛过程

从表1中可以看出，ALM-Dual 运行所需的时间和迭代次数与前两个近似梯度法相近，但从图8可以看出，ALM-dual 的收敛曲线并不像前两者一样，在刚开始时快速收敛，这或许是因为 ALM-dual 并没有使用连续化策略。

3.10 交替方向乘子法求解对偶问题

先给出原问题的对偶问题：

$$\begin{aligned} \min_{z} \quad & \frac{1}{2}\|z\|_F^2 + \langle b, z\rangle + \mathbb{I}_{\|w\|_{\infty,2}\le\mu}(w) \\ \text{s.t.} \quad & A^T z = w \end{aligned}$$

增广拉格朗日函数为：

$$L_\rho(z,w,y) = \frac{1}{2}\|z\|_F^2 + \langle b, z\rangle + \mathbb{I}_{\|w\|_{\infty,2}\le\mu}(w) + \langle y, A^T z - w\rangle + \frac{\rho}{2}\|A^T z - w\|_F^2$$

我们需要在以下几个不同的条件下求解 $\min_{w,y,z} L_\rho(z,w,y)$，进而得到 w, y, z 的迭代公式：

固定 $w = w^{(k)}, y = y^{(k)}$ 时，得到

$$z^{(k+1)} = (I_m + \rho A A^T)^{-1}(A(\rho w^{(k)} - y^{(k)}) - b)$$

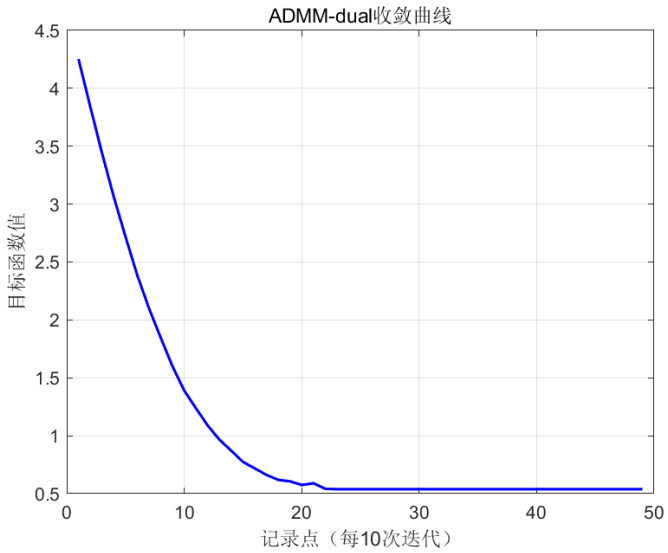


图 9: ADMM-dual 收敛过程

固定 $z = z^{(k)}, y = y^{(k)}$ 时，得到：

$$w_i^{(k+1)} = \begin{cases} v_i, & \|v_i\| \leq \mu \\ \mu \cdot \dfrac{v_i}{\|v_i\|_2}, & \|v_i\| > \mu \end{cases}$$

其中 $v = A^T z^{(k+1)} + \frac{1}{\rho} y^{(k)}$。

y 的更新为：

$$y^{(k+1)} = y^{(k)} + \rho(A^T z^{(k+1)} - w^{(k+1)})$$

此算法的运行结果如图9

从表1中可以看出，ADMM-dual 与 ALM-dual 用时相近，但前者所需的迭代次数更多。

3.11 交替方向乘子法求解原问题

引入辅助变量 z，原问题转化为：

$$\begin{aligned} \min_{x,z} \quad & \frac{1}{2}\|Ax - b\|_F^2 + \mu\|z\|_{1,2} \\ \text{s.t.} \quad & x - z = 0 \end{aligned}$$

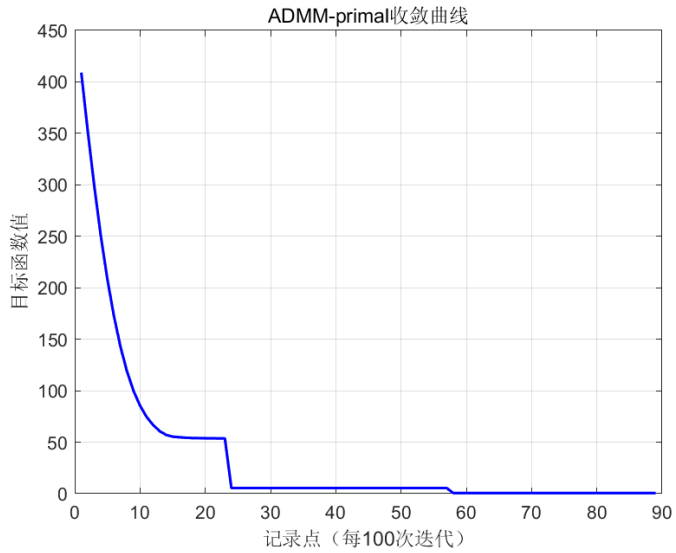


图 10: ADMM-primal 收敛过程

增广拉格朗日函数为：

$$L_\rho(x,z,y)=\frac{1}{2}\|Ax-b\|_F^2+\mu\|z\|_{1,2}+\langle y,x-z\rangle+\frac{\rho}{2}\|x-z\|_F^2$$

标准 ADMM 迭代为：

- x 子问题：

$$\begin{aligned}x^{(k+1)}=&\arg\min_x\frac{1}{2}\|Ax-b\|_F^2+\frac{\rho}{2}\|x-z^{(k)}+\frac{1}{\rho}y^{(k)}\|_F^2\\=&(A^TA+\rho I)^{-1}(A^Tb+\rho z^{(k)}-y^{(k)})\end{aligned}$$

- z 子问题：

$$\begin{aligned}z^{(k+1)}=&\arg\min_z\mu\|z\|_{1,2}+\frac{\rho}{2}\|x^{(k+1)}-z+\frac{1}{\rho}y^{(k)}\|_F^2\\=&prox_{(\mu/\rho)\|\cdot\|_{1,2}}(x^{(k+1)}+\frac{y^{(k)}}{\rho})\\z_i^{(k+1)}=&\begin{cases}(1-\dfrac{\mu}{\rho\|w_i^k\|_2})w_i^k, & \|w_i^k\|_2>\dfrac{\mu}{\rho}\\0, & otherwise\end{cases}\end{aligned}$$

- 乘子更新：

$$y^{(k+1)}=y^{(k)}+\rho(x^{(k+1)}-z^{(k+1)})$$

此算法的运行结果如图10

从表1中可以看出，ADMM-primal 的迭代次数显著多于 ALM-Dual 和 ADMM-Dual，

但运行时间差不多。从图10中可以看出，连续化策略使得 ADMM-primal 在刚开始时快速收敛，μ 变小时会出现阶梯状下降。

4 实验结果

所有算法的运行结果整合在了下面两个表中：

表 1: 不同算法的运行结果

Method	**CPU(s)**	**Iter**	**Optval**	**Sparsity**	**Err-to-exact**
CVX-Mosek	9.05	11	5.38327E-01	0.115	4.22E-05
CVX-Gurobi	4.15	15	5.38327E-01	0.114	4.19E-05
Mosek	0.34	11	5.38327E-01	0.114	4.20E-05
Gurobi	0.78	14	5.38327E-01	0.116	4.23E-05
SGD Primal	1.03	7268	5.38328E-01	0.146	5.33E-05
GD Primal	13.88	86299	5.38335E-01	0.497	5.90E-05
FGD Primal	2.71	6283	5.38327E-01	0.117	4.28E-05
ProxGD Primal	1.74	218	5.38327E-01	0.114	4.20E-05
FProxGD Primal	1.30	239	5.38327E-01	0.114	4.20E-05
ALM Dual	1.36	260	5.38327E-01	0.100	6.86E-05
ADMM Dual	0.93	497	5.38328E-01	0.100	5.02E-05
ADMM Primal	1.22	9126	5.38327E-01	0.114	4.20E-05

表 2: 不同算法的精度

Method	**Err-to-CVX-Mosek**	**Err-to-CVX-Gurobi**
CVX-Mosek	0.00E+00	3.76E-07
CVX-Gurobi	3.76E-07	0.00E+00
Mosek	2.41E-07	1.44E-07
Gurobi	2.43E-07	5.73E-07
SGD Primal	1.45E-05	1.48E-05
GD Primal	1.91E-05	1.94E-05
FGD Primal	8.19E-07	1.03E-06
ProxGD Primal	3.47E-07	5.01E-08
FProxGD Primal	3.42E-07	5.50E-08
ALM Dual	3.19E-05	3.21E-05
ADMM Dual	1.07E-05	1.09E-05
ADMM Primal	3.51E-07	6.46E-08

5 参考资料

【1】文再文老师的课程讲义与课程主页 http://faculty.bicmr.pku.edu.cn/ wenzw/opt-2025-fall.html

【2】程序作业提交指南 http://faculty.bicmr.pku.edu.cn/ wenzw/opt2015/homework5-req.pdf

【3】李锦韬学长的仓库 https://github.com/AkexStar/Algorithms-group-LASSO-problem

【4】王晨宇学长的仓库 https://github.com/cwangPKU/Optimization_Algorithms